

GPU-Accelerated TEBD for the 1D Transverse-Field Ising Model

Final Report, CME 213: Parallel Computing with CUDA, MPI and OpenMP

Hercy Shen, Chiling Han, Xin Wei — June 8, 2026

1. Project description

We simulate the **real-time quench dynamics** of the one-dimensional transverse-field Ising model (TFIM), a canonical quantum many-body system, with the Hamiltonian described as:

$$H = -J \sum_i \sigma_i^x \sigma_{i+1}^x - g \sum_i \sigma_i^z. \quad (1)$$

A chain of L spin- $\frac{1}{2}$ sites lives in a Hilbert space of dimension 2^L , so storing the wavefunction exactly is impossible beyond $L \approx 30$. The key physical fact that makes simulation tractable is that low-energy and short-time states of 1D local Hamiltonians have *bounded entanglement*, and can be compressed into a *Matrix Product State (MPS)*: the 2^L amplitudes are factored into L small rank-3 tensors connected by *bond* indices of dimension χ . The MPS stores only $O(L\chi^2 d)$ numbers ($d = 2$), where χ grows with the entanglement across each cut.

Computation. We evolve the state with *Time-Evolving Block Decimation (TEBD)*: the propagator e^{-iHt} is Suzuki–Trotter split into nearest-neighbour two-site gates $U_b = e^{-i\Delta t H_b}$, and each Trotter step applies $L - 1$ gates. Applying one gate is one small complex SVD that re-compresses the two-site tensor back to bond dimension χ ; this SVD takes $> 99.6\%$ of GPU kernel time and is the target of all parallel work here.

- **Inputs:** chain length L ; couplings (J, g) ; an initial product state; Trotter step dt ; number of steps N ; max bond dimension $\chi_{\max}$.
- **Outputs:** mean field magnetizations $\langle \sigma_i^z \rangle$, $\langle \sigma_i^x \rangle$, two-point correlations $\langle \sigma_i^x \sigma_j^x \rangle$, the bipartite von-Neumann entanglement entropy, and the bond-dimension profile $\chi(t)$.
- **Assumptions:** open boundary conditions, local dimension $d = 2$.

Why parallelism. Entanglement grows under a quench, so $\chi(t)$ climbs over time (Fig. ??) until it saturates $\chi_{\max}$. Each bond update requires a small dense SVD whose cost scales roughly as $O(\chi^3)$, so doubling the effective bond dimension gives an $8\times$ increase in work. On a single GPU, each bond update runs as device kernels: our CUDA kernels handle the gate contraction, layout conversion, and reconstruction of the MPS tensors around the SVD, while cuSOLVER performs the SVD in the production path.

Across GPUs we use task parallelism: each MPI rank owns one GPU and runs independent TEBD evolutions for different (J, g) points. This matches the structure of the parameter sweep and turns many independent simulations into a shorter wall-clock study.

2. Algorithms and state of the art

Core algorithm. We implement TEBD on a *right-canonical MPS* in Vidal form. The two-site update is: (1) contract the gate with the two-site tensor; (2) SVD the reshaped $(\chi_L d) \times (d \chi_R)$ matrix; (3) truncate to the largest $\chi_{\max}$ singular values and renormalize; (4) restore MPS form.

State of the art. Mature libraries (*TeNPy*, *ITensor*, *quimb*, *TensorNetwork*) are CPU/Python-centric and general; on GPUs, NVIDIA’s *cuTensorNet* (cuQuantum) offers library-level batched contraction and SVD. The algorithmic variants relevant to our problem are the SVD engine, the time integrator (Trotter–Suzuki), and the parallel decomposition.

Our choice and how it compares. We do *not* try to beat cuTensorNet on throughput. Instead we build an instrumented TEBD that times each kernel separately, so we can attribute time to gate

contraction, SVD, truncation, and PCIe traffic rather than have a library hide it.

3. Parallelization strategy (CUDA + MPI)

3.1. The central difficulty and the decomposition we chose

A single TEBD chain has *limited spatial parallelism*. Within one Trotter sub-step the even bonds $\{0, 2, 4, \dots\}$ and odd bonds $\{1, 3, 5, \dots\}$ touch disjoint tensors and are independent ($\sim (L-1)/2$ -way), but the even and odd sub-steps are *sequential*, and a spatial decomposition that splits the chain across ranks would require one MPI round-trip per sub-step to exchange the boundary Schmidt spectrum. We benchmark up to $L = 80$, a study-scope choice (short runs), not a memory limit (the MPS holds only $O(L\chi^2 d)$ numbers, tens of MB); the per-step compute is far too small to hide that latency.

We therefore use *task parallelism*: each MPI rank owns one GPU and runs complete, independent TEBD evolutions for a set of (J, g) points in the phase diagram, with zero in-loop communication, as production parameter studies do. We rejected two alternatives: (a) spatial chain decomposition, infeasible at small L due to per-sub-step synchronization; (b) replicated-parameter plus distributed chain, which needs per-bond MPI barriers.

3.2. Single-GPU bond-update pipeline

The bond update is implemented as a five-stage device pipeline. With $m = \chi_L d$, $n = d\chi_R$, $K = \min(m, n)$ and block shapes for $d = 2$:

#	Stage	Kernel / call	Notes
1	gate contraction	<code>gate_contract_kernel<2></code>	gate in shared memory ; <code>#pragma unroll</code>
2	row→col layout	<code>row_to_col_kernel</code>	coalesced reads for column-major SVD
3	SVD	<code>cusolverDnZgesvdj</code>	Jacobi, $K = \min(m, n)$, <code>lwork</code> re-queried per call
4	Vidal restore (L)	<code>vidal_restore_left_kernel<2></code>	$\text{new}B_i[a, p, b] = U[ad+p, b] S[b]/\Lambda_a$
5	Vidal restore (R)	<code>vidal_restore_right_kernel<2></code>	in-place conjugation of V

Every stage is bracketed by CUDA events so `bench_breakdown` can report per-stage time. The SVD is $> 99.6\%$ of kernel time (§5); all other kernels are $< 0.4\%$.

3.3. Persistent device-resident MPS and streams

- **Persistent device MPS (GpuMps).** All L B-tensors and Schmidt spectra stay device-resident for the whole evolution, dropping profiled H2D/D2H from tens of MB to ~ 0.2 MB; buffers are pre-allocated once (`GpuTebdWorkspace`), so there is zero `cudaMalloc` during evolution.
- **Streams within a sub-step.** `tebd_step_gpu_persistent` accepts n workspaces with independent CUDA streams and `cuSOLVER` handles. However, truncation requires a `cudaStreamSynchronize` mid-bond (to read singular values host-side), serializing bonds. Measured stream speedup is negligible (Table 1), as the performance model predicts.

3.4. MPI decomposition and communication pattern

Each rank calls `cudaSetDevice(rank % n_gpus)`, builds its (J, g) points locally (no scatter), and runs independent evolutions; the TEBD loop contains no MPI calls. Only at the end does rank 0 collect results: an `MPI_Gather` of one `int` per rank (record counts), then an `MPI_Gatherv` of variable-length result records (timing, $\chi_{\max}$, S_{mid} , and $2L + L^2$ observable values).

4. Performance model

We build a quantitative, falsifiable model from four pieces.

(a) **Per-bond compute and the latency/compute crossover.** The dominant cost is the SVD of a $2\chi_a \times 2\chi_a$ complex matrix. One-sided Jacobi does $O(\chi_a^3)$ flops per sweep over $\sim 5\text{--}10$ sweeps, plus a fixed overhead t_0 per bond:

$$T_{\text{step}}(L, \chi_a) \approx (L - 1) (t_0 + c\chi_a^3). \quad (2)$$

Predictions: (i) when $c\chi_a^3 \ll t_0$ the code is **latency-bound**: T is linear in L and flat in χ_{max} ; (ii) when $c\chi_a^3 \gg t_0$ it becomes **work-dominated**: $T \propto \chi_a^3$. Our runs sit in branch (i) (§6).

(b) **Roofline / arithmetic intensity.** Each Jacobi sweep does $O(\chi_a^3)$ flops on an $O(\chi_a^2)$ -word matrix that fits in L2, so intensity $I = O(\chi_a)$ grows with χ_a and the matrix is cache-resident.

Prediction: the SVD is never bandwidth-bound; at the small realized χ_a the limiter is kernel-launch and dispatch latency, not bandwidth or peak compute, and the compute roof is reached only at χ_a far beyond our range.

(c) **Communication model.** The only message is the final **Gatherv** of $\lesssim 10$ KB. **Prediction:** communication fraction $< 10^{-4}$, invisible in a profiler.

(d) **Parallel efficiency (task-parallel load balance).** With P ranks and per-pair costs t_i :

$$E = \frac{\sum_i t_i}{P \max_r \sum_{i \in r} t_i}. \quad (3)$$

Pair cost rises steeply with J (more entanglement $\Rightarrow$ larger $\chi_a \Rightarrow O(\chi_a^3)$).

5. Benchmarking and instrumentation

Instrumentation. Per-stage GPU time measured with *CUDA events* (**bench_breakdown**); full-step wall time via host timers (**bench_gpu**); SVD-engine comparison via **bench_svd**; end-to-end timelines with *Nsight Systems* (**nsys profile --stats=true**), single-GPU and under **mpirun -n 4**.

Single-GPU full-step timing (**bench_gpu**, $L = 20$, 20 steps, **gpu-turing**):

χ_{max}	CPU (s)	M3 GPU (s)	Persistent (s)	Streamed (s)	CPU/Persistent
16	2.049	5.372	5.180	5.183	0.40
32	9.601	13.196	12.992	13.064	0.74
48	20.073	17.428	17.075	17.079	1.18
64	24.099	18.250	17.709	17.745	1.36
96	24.484	18.269	17.722	17.854	1.38
128	24.533	18.334	17.848	17.853	1.37

Table 1. GPU overtakes CPU at $\chi_{\text{max}} \geq 48$; persistent MPS is 1–3% faster than M3 host-staging; four streams add nothing.

Per-stage kernel-time breakdown (**bench_breakdown** / **Nsight cuda_gpu_kern_sum**):

GPU kernel group	Share	Note
cuSOLVER gesvdbj + svd.col/row_rotate	96.4%	top-3 Jacobi SVD kernels
all remaining cuSOLVER helpers	3.2%	sort, scale, QR, etc.
gate_contract_kernel (ours)	0.09%	
vidal_restore left+right (ours)	0.13%	
theta2 + ss_inv kernels (ours)	0.11%	persistent path
row_to_col_kernel (ours)	0.06%	
MPI communication	$< 0.01\%$	Gatherv of ~ 10 KB at end

Table 2. cuSOLVER Jacobi is $> 99.6\%$ of GPU kernel time; all custom kernels together $< 0.4\%$.

Model vs. measurement. The crossover study (`bench_update`, Fig. ??) confirms prediction (a): at TFIM-realized $\chi_a \approx 41$ the GPU is latency-bound (flat across $\chi_{\max} = 64\text{--}128$ in Table 1), crossing the CPU near $\chi \approx 96$ and reaching $\sim 5\times$ at $\chi = 128$. Nsight confirms model (c): no MPI call appears in the timeline.

6. Bottleneck analysis

The bottleneck has three parts, each tied to hardware:

- **Not memory-bound.** cuSOLVER’s Jacobi kernels are $> 99.6\%$ of GPU kernel time (Table 2). A roofline estimate rules out bandwidth: the $2\chi_a \times 2\chi_a$ matrix is L2-resident, so intensity $I \approx 9n/16$ flops/byte grows with χ_a and at $\chi_a = 64$ reaches $I \approx 72$, $\sim 95\times$ past the ridge (≈ 0.76). But being past the ridge only identifies *compute* (not bandwidth) as the binding ceiling *if* the kernel saturates the GPU; it does not say the kernel reaches it.
- **Latency-bound at the realized sizes.** Entanglement fixes the realized $\chi_a \approx 41$ for this quench (≤ 64 across the sweep), distinct from the budget $\chi_{\max}$; the resulting SVD is too small in flops and parallelism to saturate the SMs. Table 1 bears this out in the GPU/CPU ratio: the GPU *loses* at small $\chi_{\max}$ (truncation caps χ_a tiny and launch/dispatch overhead t_0 dominates), then wins only $\sim 1.4\times$ once $\chi_{\max} \geq 48$ and *plateaus* as χ_a saturates. A compute-bound kernel would keep speeding up with χ ; ours plateaus.
- **Multi-GPU: load imbalance, not communication.** Computation is $> 99\%$ of wall time; the final `Gatherv` is $< 0.01\%$ (Table 2). The distributed bottleneck is purely **load imbalance**: round-robin gives one rank the four hardest ($J = 2.0$) pairs, which run $2.2\times$ longer than the cheapest, explaining the $\sim 19\%$ efficiency gap at $P = 4$ (§8). The heatmap (Fig. ??) shows the underlying χ_a spread (21 to a saturated 64, a $27\times$ range in SVD flops).

Custom-kernel profiling (Nsight Compute). `ncu` on our from-scratch Jacobi kernel ($\chi_{\max} = 128$, 256×256 complex; Appendix C) shows it is neither compute-bound (0.42% SM throughput) nor bandwidth-bound (0.00% DRAM, L2 $\approx 100\%$): it is **under-decomposed**. A single 128-thread block leaves 1 of 72 SMs active (the GPU $\approx 99.8\%$ idle), and its four warps cannot hide the per-rotation `__syncthreads` barriers (41.6% of stalls), so it is latency-bound. (The 12.5% occupancy reflects the one-block grid, not a register or shared-memory cap.) cuSOLVER keeps far more of the device busy and stays $1.2\text{--}2.2\times$ faster (Table 3). This is the same shortage of per-chain parallelism behind our task-parallel design (§3): even batching the $\sim (L - 1)/2$ bonds would not fill 72 SMs at $L \leq 80$.

7. Algorithmic variants and their effect on performance

We explored four variants; each shifts the bottleneck differently.

1. **Host-staging (M3) $\rightarrow$ persistent device MPS.** Eliminating per-bond H2D/D2H cuts PCIe traffic from tens of MB to ~ 0.2 MB for a steady $1\text{--}3\%$ speedup (Table 1), small because the SVD dominates.
2. **Single stream $\rightarrow$ 4 streams.** Negligible (Table 1, Fig. ??). The model predicted it; profiling showed why: host-side truncation serializes bonds.
3. **cuSOLVER gesvdj vs. custom one-sided Jacobi SVD.** Our hand-written Jacobi kernel is **$1.2\text{--}2.2\times$ slower** than cuSOLVER and agrees to $\lesssim 10^{-12}$ (Table 3, Fig. ??). Both paths plateau once $\chi_{\max}$ exceeds the realized $\chi_a \approx 41\text{--}64$ (cuSOLVER time changes $< 1\%$ from $\chi_{\max} = 64$ to 256), a direct consequence of the latency-bound regime: SVD matrix size is set by χ_a , not $\chi_{\max}$.
4. **Trotter order ($1/2/4$).** Higher order trades more gates per step for $O(dt^p)$ accuracy; order 4 (Forest–Ruth) reaches the ED-validation floor ($\sim 10^{-6}$, §9) at larger dt .

$\chi_{\max}$	cuSOLVER (s)	Custom Jacobi (s)	Ratio	$\max \Delta\sigma $
16	7.110	8.184	0.869	4.74×10^{-14}
32	17.352	27.089	0.641	5.84×10^{-13}
48	22.688	46.056	0.493	9.96×10^{-13}
64	23.751	52.160	0.455	1.00×10^{-12}
128	23.754	52.208	0.455	1.00×10^{-12}
256	23.985	52.435	0.457	1.00×10^{-12}

Table 3. *bench_svd*: cuSOLVER vs. custom Jacobi, $L = 20$, 20 steps. Both paths plateau above $\chi_{\max} = 64$; the wide-matrix adjoint fix reduced error from $O(10^{-1})$ to $\lesssim 10^{-12}$.

8. Scalability analysis

We study scaling of the task-parallel sweep on `gpu-turing`, using total rank wall time $T(P) = \max_r \sum_{i \in r} t_i$.

Mode	Ranks	Pairs/rank	Time (s)	Speedup	Efficiency
strong	1	16	321.6	1.00	100%
strong	2	8	184.3	1.75	87%
strong	4	4	99.1	3.24	81%
weak	1	4	17.1	n/a	100%
weak	2	4	59.1	n/a	29%
weak	4	4	103.5	n/a	17%

Table 4. MPI strong and weak scaling. Strong speedup relative to $P = 1$ wall time (321.6 s).

Strong scaling. Speedup reaches $3.24\times$ at $P = 4$ (81% efficiency, Fig. ??). The 19% loss is exactly the load imbalance the model predicted: round-robin hands rank 3 the four $J = 2.0$ pairs (highest entanglement), which run $\sim 2.2\times$ longer than rank 0’s cheap $J = 0.25$ pairs. Communication is $< 0.01\%$.

Weak scaling. Efficiency falls sharply (29% at $P = 2$, 17% at $P = 4$), but this is *not* a communication or synchronization failure. The weak mode hands each added rank *higher- J* pairs, which are intrinsically more expensive (2–6 $\times$ the cost of the $P = 1$ pairs). True constant-work weak scaling would require balancing by *estimated entanglement* rather than pair count.

Phase-diagram structure. Figure ?? shows the end-of-evolution mid-chain entropy and realized χ_a across the 4×4 grid: entropy peaks near the critical point ($J \approx g$) and saturates $\chi_{\max} = 64$ at most high- J points, while the paramagnetic corner reaches only $\chi_a \approx 21$ –45. This 2–6 $\times$ spread in χ_a sets the per-pair SVD cost and explains the load-imbalance gap.

Observables over the phase diagram. The sweep also collects GPU-computed $\langle \sigma_i^z \rangle$, $\langle \sigma_i^x \rangle$, and $\langle \sigma_i^x \sigma_j^x \rangle$ for every pair (Fig. ??). $\langle \sigma_i^x \rangle$ is zero by parity (H commutes with $\Pi = \bigotimes \sigma_i^z$, under which $\sigma_i^x \rightarrow -\sigma_i^x$), and the ferromagnetic corner (large J , small g) shows the largest NN correlations and smallest $|\langle \sigma_i^z \rangle|$, as expected.

9. Correctness and verification

We verify correctness at four independent levels:

1. **Unit tests.** 54 Google Test cases across 6 binaries cover tensor ops, the hand-written GEMM/GEMV/SVD/expm/contraction/kron, MPS observables, the TFI model (Hermiticity, sum rules), and TEBD invariants (norm preservation, S^z conservation, entropy growth, Trotter-order convergence). All pass at 10^{-12} (exact ops) / 10^{-10} (SVD-dependent).
2. **Analytical / exact-diagonalization oracle.** A full $2^L \times 2^L$ state-vector evolution validates

TEBD on small systems. At $L = 8$, $dt = 0.05$, $t = 2$, order 4, $\chi_{\max} = 64$: max error 5.1×10^{-6} in $\langle \sigma^z \rangle$, 4.7×10^{-6} in $\langle \sigma^x \sigma^x \rangle$, 2.2×10^{-6} in entropy, and final fidelity loss 4.3×10^{-11} . The residual is dominated by the $O(dt^4)$ Trotter error; it sits at the Trotter floor from the first step onward (Fig. ??).

3. **CPU↔GPU agreement.** `validate_gpu` and `validate_persistent` run GPU and CPU reference side-by-side for 30 steps. Max discrepancies are 3.53×10^{-13} and 3.60×10^{-13} , at the floor where CPU Jacobi and cuSOLVER `gesvdj` agree on singular values. Custom Jacobi agrees to $\lesssim 10^{-12}$ (Table 3).
4. **Multi-rank consistency.** 1-, 2-, and 4-rank sweeps produce *identical* $(J, g) \rightarrow (\chi_{\max}, S_{\text{mid}})$ records, and 1-stream and 4-stream persistent configurations agree to every printed digit.

10. Discussion, limitations, and future work

What worked, and the largest payoffs. Task-parallel MPI with zero in-loop communication gave 81% strong-scaling efficiency (load-imbalance-limited), and the persistent device MPS removed the M3 PCIe bottleneck.

What was not worth the complexity. *CUDA streams* gave negligible speedup (host truncation serializes bonds). Our from-scratch *custom Jacobi SVD* (the required hand-written kernel) is 1.2–2.2× slower than cuSOLVER, with `ncu` explaining why (§6).

Limitations and future directions.

- **Load-balanced assignment.** Round-robin ignores the $\sim 2\text{--}6\times$ spread across (J, g) ; sorting by estimated χ_a and bin-packing should lift efficiency toward $\gtrsim 95\%$ and fix the weak-scaling collapse.
- **Device-side truncation.** A GPU top- k selection would remove the mid-pipeline host sync and let streams overlap bonds.
- **Spatial decomposition for $L \gg 100$.** Needed only when one evolution exceeds GPU memory or there are too few (J, g) points to fill the ranks; it would add an `MPI_Allgather` of the boundary spectrum per sub-step.

Connection to lecture. The project touches the GPU memory hierarchy (shared memory, coalesced layouts), the roofline model (the cache-resident SVD is never bandwidth-bound), the warp-execution model (the single-block Jacobi is latency/barrier-bound, not compute-bound), MPI collectives, and Amdahl/load-balance analysis.

Hardware/software. Benchmarks ran on `gpu-turing` (Quadro RTX 6000, `sm_75`, CUDA 12.3, OpenMPI); development also used a local RTX 5090 (`sm_120`) and a one-core `g++` baseline.

References

1. G. Vidal, *Efficient classical simulation of slightly entangled quantum computations*, PRL 91, 147902 (2003); *Efficient simulation of one-dimensional quantum many-body systems*, PRL 93, 040502 (2004).
2. U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, Ann. Phys. 326, 96 (2011).
3. R. P. Brent and F. T. Luk, *The solution of singular-value and symmetric eigenvalue problems on multiprocessor arrays*, SIAM J. Sci. Stat. Comput. (1985).
4. N. Hatano and M. Suzuki, *Finding exponential product formulas of higher orders* (Suzuki–Trotter decomposition).
5. NVIDIA cuSOLVER / cuQuantum (cuTensorNet) documentation.
6. TeNPy, ITensor, quimb (open-source tensor-network libraries).

A. Debugging notes

Three subtle bugs were caught with `compute-sanitizer` and auditing: (i) a gauge bug from `cuSOLVER` returning V (not V^H) in column-major, fixed by an explicit row $\rightarrow$ col transpose and conjugation in the right-restore kernel; (ii) a gate shared-memory load that zero-padded when a block had < 16 threads, fixed with a cooperative strided load; (iii) an out-of-bounds write from `gesvdj_bufferSize` being non-monotone in (m, n) , fixed by re-querying `lwork` per call.

B. Supplementary figures

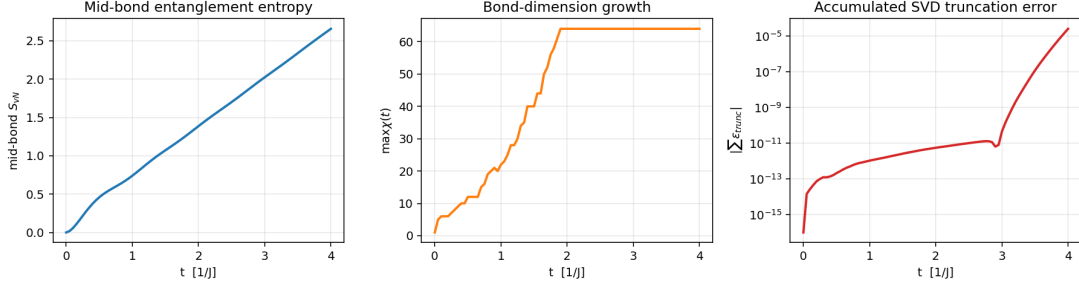


Figure 1: Reference $L = 20$ TFIM quench ($J = g = 1$, order 4, $\chi_{\max} = 64$). Entanglement growth drives bond-dimension growth, which sets the SVD sizes that dominate runtime.

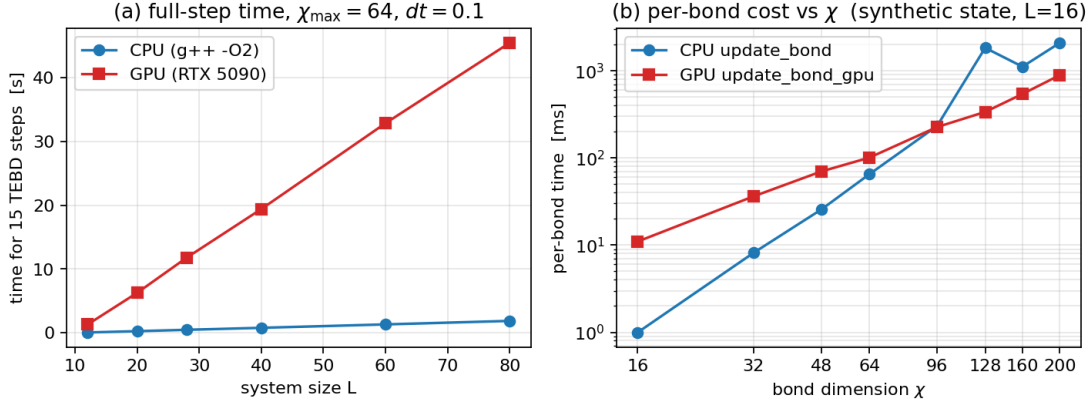


Figure 2: (a) Wall time vs. L at $\chi_{\max} = 64$: linear, ~ 1.3 ms/bond, latency-bound as predicted. (b) Per-call `update_bond` vs. χ : CPU/GPU crossover near $\chi \approx 96$ –128.

C. Custom-kernel profile (Nsight Compute)

Nsight Compute (`ncu`) on the custom one-sided Jacobi kernel at $\chi_{\max} = 128$ (256×256 complex matrix, launched `<<<1,128>>>`):

TFIM phase-diagram sweep ($L = 20$, $\chi_{\max} = 64$, 20 steps)

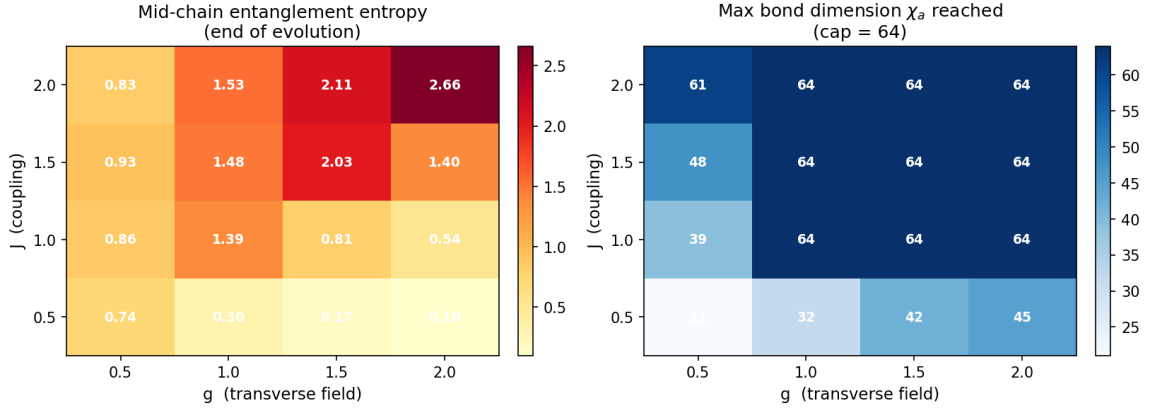


Figure 3: Left: final mid-chain entropy S_{mid} . Right: max realized bond dimension χ_a . High- J points saturate $\chi_{\max} = 64$ and dominate runtime; the paramagnetic corner (small J , large g) is cheap.

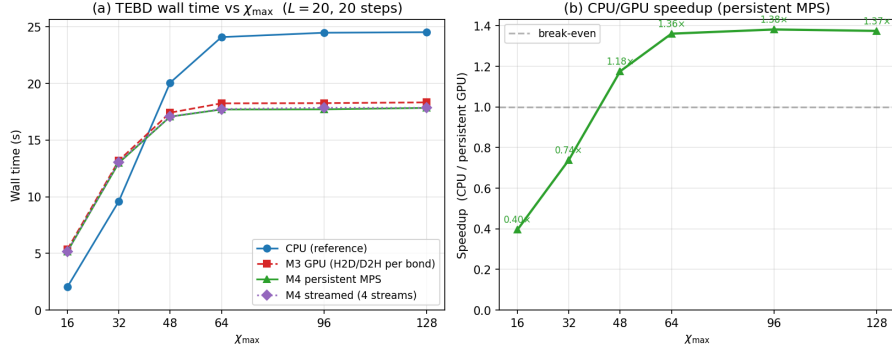


Figure 4: Single-GPU full-step timing vs. $\chi_{\max}$ (companion to Table 1).

Metric	Value	Implication
Compute (SM) throughput	0.42%	not compute-bound
DRAM throughput	0.00% (10.8 MB/s)	not bandwidth-bound
L2 / L1 hit rate	$\approx 100\%$ / 92.6%	matrix is cache-resident
Achieved occupancy	12.5% (4/32 warps)	one block (not a resource cap)
SMs active	1 of 72	grid is a single block
Full waves across SMs	0.0	cannot fill one device wave
Warp cycles per issued inst.	22.1	long gaps between issues
Top stall (41.6%)	CTA barrier	idle at <code>__syncthreads</code>
Avg active threads/warp	29.2 (18.1 pred.)	minor lane divergence

Table 5. The custom kernel is neither compute- nor memory-bound; it is under-decomposed and latency-bound. A single 128-thread block uses $\approx 0.17\%$ of the device (12.5% occupancy $\times 1/72$ SMs), leaving the GPU $\approx 99.8\%$ idle, with 41.6% of stall cycles at `__syncthreads`. The 12.5% occupancy reflects the one-block grid, not a register or shared-memory cap (theoretical occupancy is 100%).

Why the barriers dominate. The custom Jacobi SVD carries significant synchronization overhead. For each column-pair rotation, threads first compute partial dot products and norms, then reduce them in shared memory to obtain the rotation parameters; these reductions need `__syncthreads()` barriers so that all partial sums are complete before the next reduction stage reads them. After one

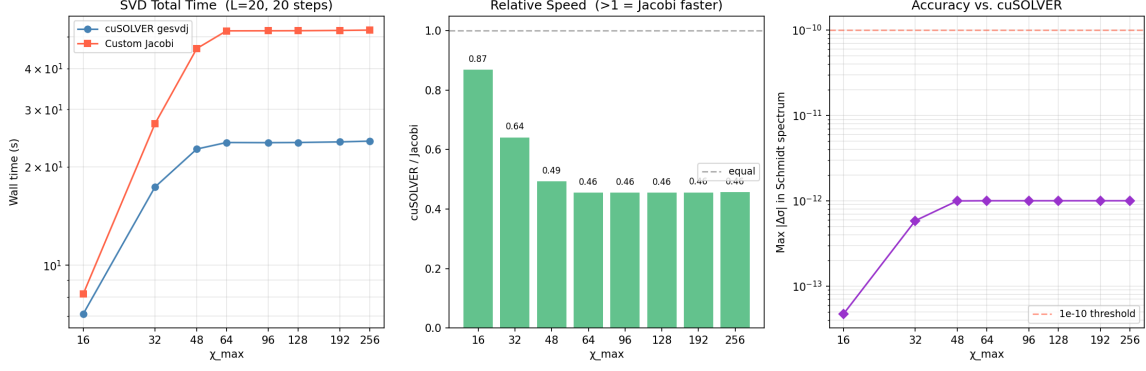


Figure 5: Custom Jacobi SVD vs. cuSOLVER `gesvdj`: time ratio (center) and Schmidt-spectrum error (right). Both plateau above $\chi_{\max} = 64$ (companion to Table 3).

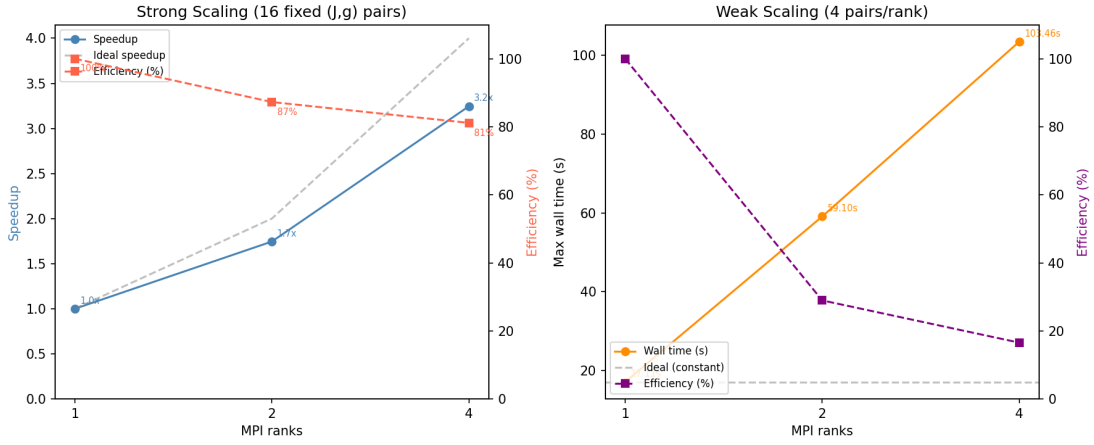


Figure 6: Strong-scaling speedup (left) and weak-scaling wall time (right) (companion to Table 4).

thread computes the rotation angle, another barrier is required before the rest of the block uses that angle to update the two columns. Since a sweep applies many such rotations sequentially and each small SVD exposes only a few warps of work, there is little independent work available to hide the barrier latency.

TFIM observables ($L = 20$, $\chi_{\max} = 64$, 20 steps)

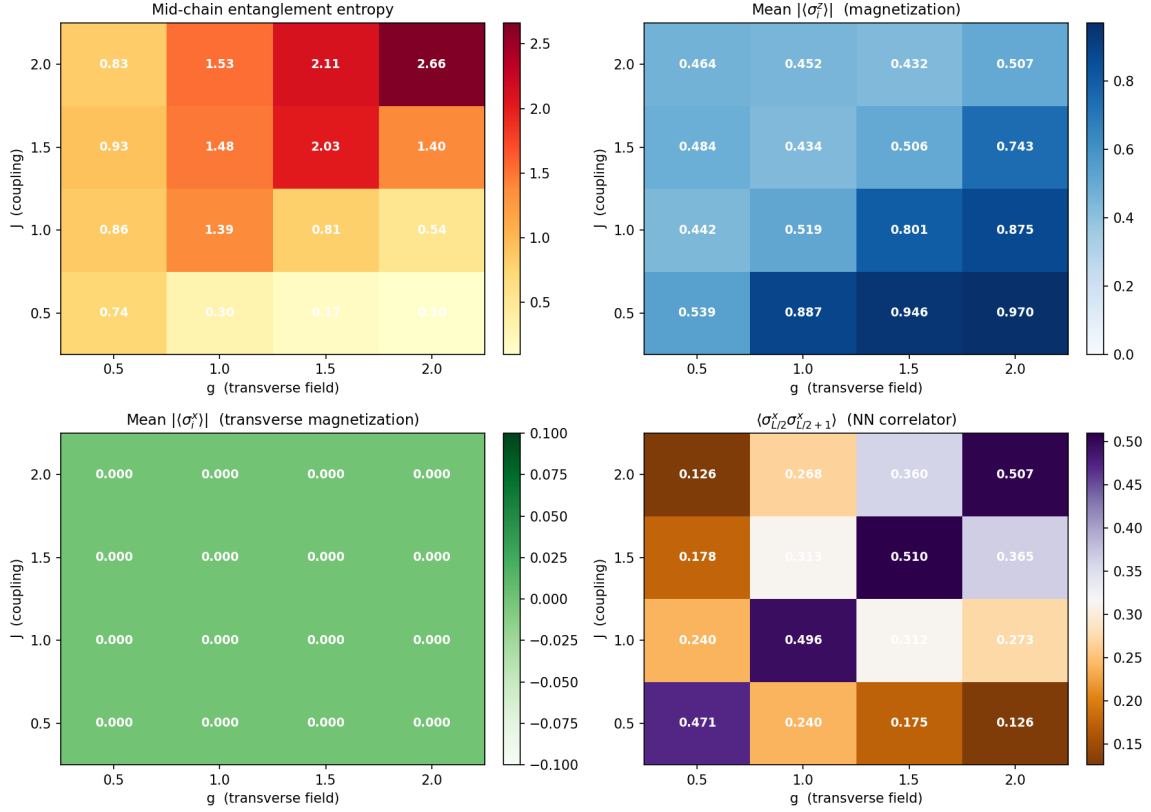


Figure 7: GPU-computed observables over the (J, g) phase diagram ($L = 20$, $\chi_{\max} = 64$, 20 Trotter steps at $dt = 0.1$). Top-right: mean $|\langle \sigma_i^z \rangle|$. Bottom-left: mean $|\langle \sigma_i^x \rangle| = 0$ (zero by parity). Bottom-right: nearest-neighbour x -correlator.

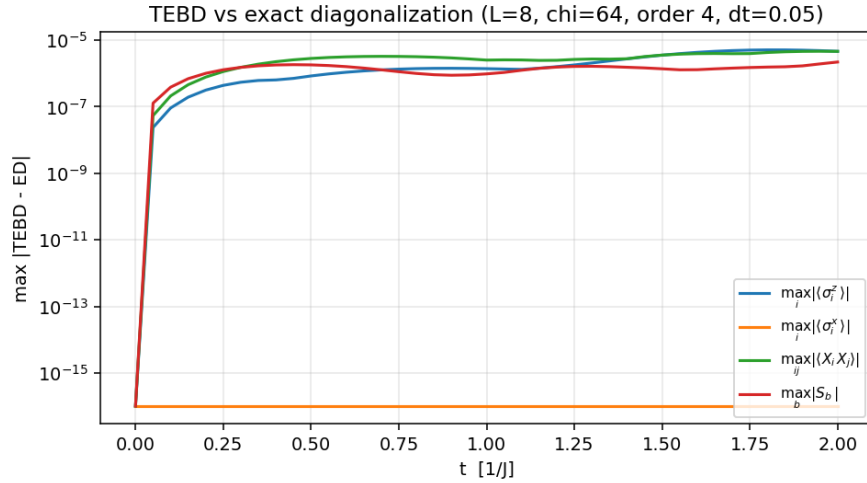


Figure 8: TEBD vs. exact diagonalization, per-step max error. Final fidelity loss 4.3×10^{-11} .